

Optional Task

Name : Shivam Ramakant Deshmane

Domain : AI & ML

College : T.B Girwalkar Polytechnic Ambajogai

[1]

✓ 4s

```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.naive_bayes import GaussianNB

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

import joblib
```

[2]

✓ 0s

```
df = pd.read_csv("StudentsPerformance.csv")
```

```
df.head()
```

▼

...

	gender	race/ethnicity	parental level of education	lunch	test preparation course	math score	reading score	writing score
0	female	group B	bachelor's degree	standard	none	72	72	74
1	female	group C	some college	standard	completed	69	90	88
2	female	group B	master's degree	standard	none	90	95	93
3	male	group A	associate's degree	free/reduced	none	47	57	44
4	male	group C	some college	standard	none	76	78	75



[3]

✓ 0s

 df.info()

▼

... <class 'pandas.core.frame.DataFrame'>

RangeIndex: 1000 entries, 0 to 999

Data columns (total 8 columns):

#	Column	Non-Null Count	Dtype
0	gender	1000 non-null	object
1	race/ethnicity	1000 non-null	object
2	parental level of education	1000 non-null	object
3	lunch	1000 non-null	object
4	test preparation course	1000 non-null	object
5	math score	1000 non-null	int64
6	reading score	1000 non-null	int64
7	writing score	1000 non-null	int64

dtypes: int64(3), object(5)

memory usage: 62.6+ KB

[4]

✓ 0s

df.shape

▼

(1000, 8)

[5]

✓ 0s

 df.describe()

▼

...

	math score	reading score	writing score
count	1000.00000	1000.000000	1000.000000
mean	66.08900	69.169000	68.054000
std	15.16308	14.600192	15.195657
min	0.00000	17.000000	10.000000
25%	57.00000	59.000000	57.750000
50%	66.00000	70.000000	69.000000
75%	77.00000	79.000000	79.000000
max	100.00000	100.000000	100.000000



[6]
✓ 0s



```
df.isnull().sum()
```

▼

```
...      0
gender      0
race/ethnicity  0
parental level of education  0
lunch      0
test preparation course  0
math score  0
reading score  0
writing score  0
```

dtype: int64

[7]
✓ 0s

```
df.duplicated().sum()
```

▼

```
np.int64(0)
```

[10]
✓ 0s

```
df.tail()
```

▼

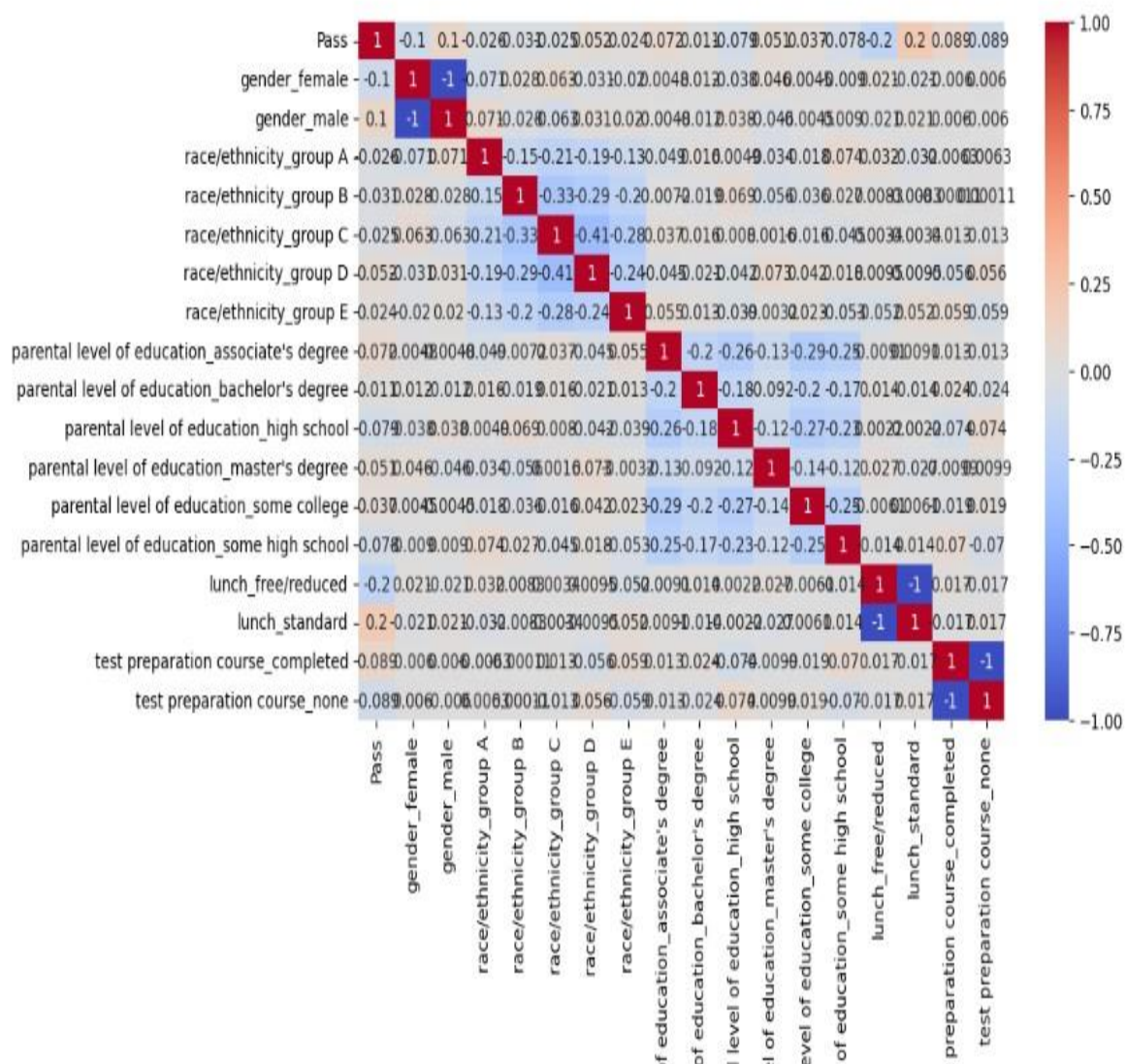
	gender	race/ethnicity	parental level of education	lunch	test preparation course	Pass
995	female	group E	master's degree	standard	completed	1
996	male	group C	high school	free/reduced	none	1
997	female	group C	high school	free/reduced	completed	1
998	female	group D	some college	standard	completed	1
999	female	group D	some college	free/reduced	none	1



[11]
✓ 0s

```
temp = pd.get_dummies(df)
```

...



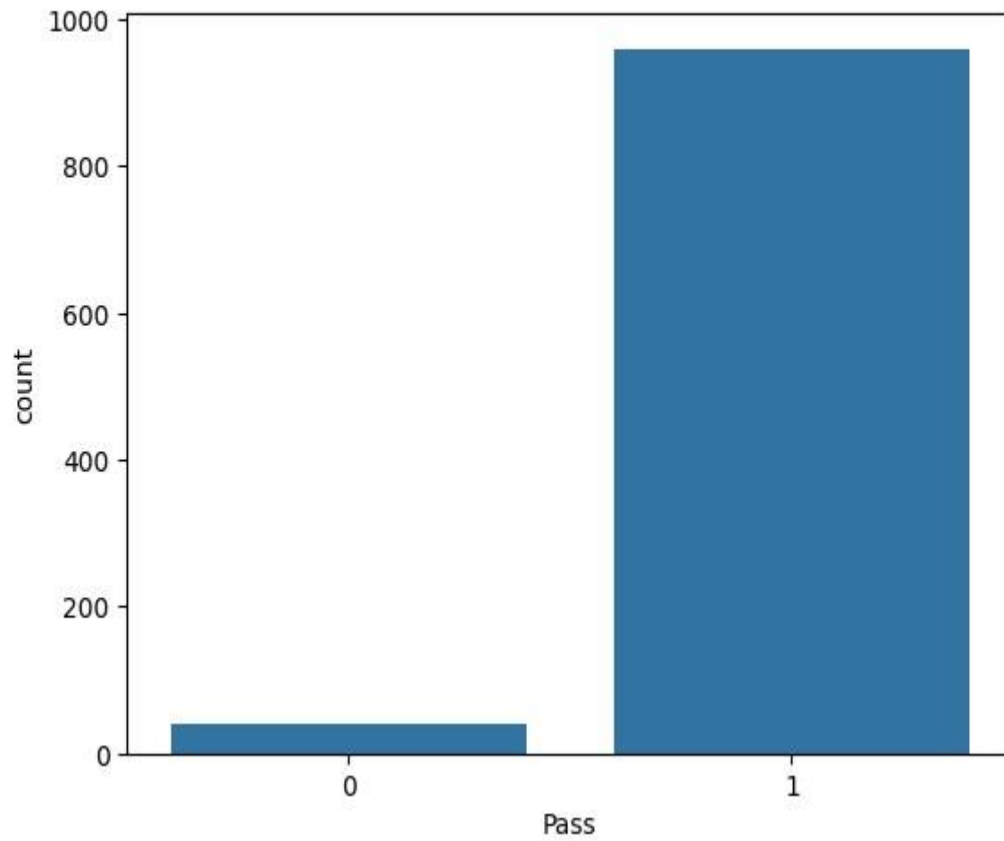
[13]

✓ Os

```
sns.countplot(x="Pass",data=df)  
plt.show()
```

▼

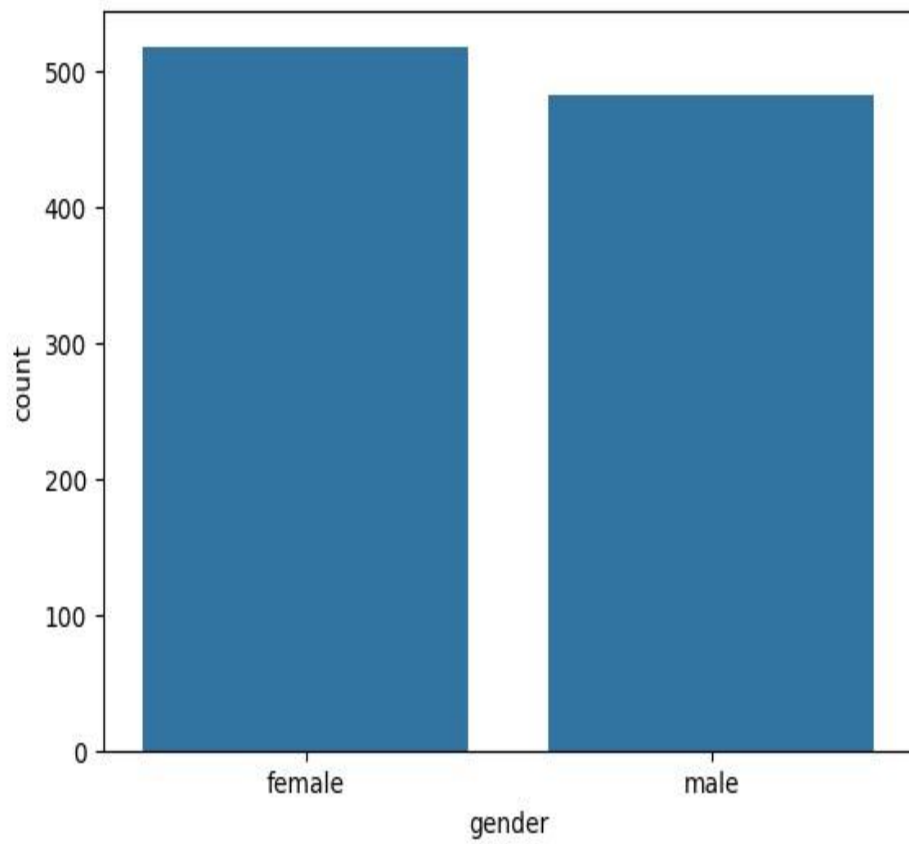
...

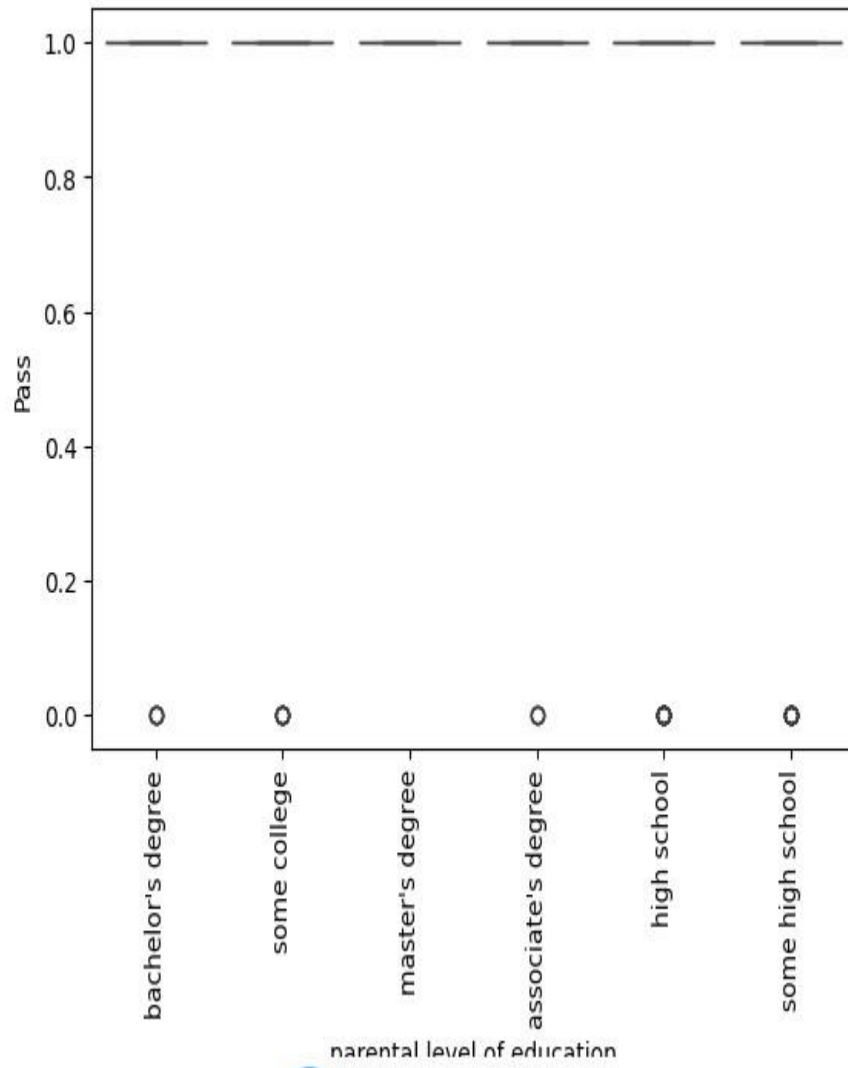


14]
/ 0s

```
sns.countplot(x="gender",data=df)  
plt.show()
```

▼





[16]
✓ 0s `df = pd.get_dummies(df, drop_first=True)`

[17]
✓ 0s `X = df.drop("Pass", axis=1)`

`y = df["Pass"]`

[18]
✓ 0s `X_train, X_test, y_train, y_test = train_test_split(
 X,
 y,
 test_size=0.2,
 random_state=42
)`

[19]
✓ 0s  `scaler = StandardScaler()`

`X_train = scaler.fit_transform(X_train)`

`X_test = scaler.transform(X_test)`

+ Code

+ Text

[20]
✓ 0s `lr = LogisticRegression()`

`lr.fit(X_train, y_train)`

`pred_lr = lr.predict(X_test)`

[21]

✓ 0s

```
knn = KNeighborsClassifier()

knn.fit(X_train,y_train)

pred_knn = knn.predict(X_test)
```

[22]

✓ 0s



```
dt = DecisionTreeClassifier(random_state=42)

dt.fit(X_train,y_train)

pred_dt = dt.predict(X_test)
```

[23]

✓ 0s

```
rf = RandomForestClassifier(random_state=42)

rf.fit(X_train,y_train)

pred_rf = rf.predict(X_test)
```

[24]

✓ 0s

```
svm = SVC()

svm.fit(X_train,y_train)

pred_svm = svm.predict(X_test)
```

	Model	Accuracy	Precision	Recall	F1 Score
0	Logistic Regression	0.945	0.945000	1.000000	0.971722
1	KNN	0.940	0.944724	0.994709	0.969072
2	Decision Tree	0.935	0.953608	0.978836	0.966057
3	Random Forest	0.940	0.953846	0.984127	0.968750
4	SVM	0.945	0.945000	1.000000	0.971722
5	Naive Bayes	0.145	1.000000	0.095238	0.173913



[29]

✓ Os

```
best_model = grid.best_estimator_
```

[31]

✓ Os

```
joblib.dump(best_model, "student_model.pkl")
```

▼

```
['student_model.pkl']
```

[32]

✓ Os

```
joblib.dump(scaler, "scaler.pkl")
```

▼

```
['scaler.pkl']
```

[33]

✓ Os

```
joblib.dump(X.columns, "columns.pkl")
```

▼

```
... ['columns.pkl']
```



[34]

✓ Os

```
model = joblib.load("student_model.pkl")
```